Optionals

We have briefly mentioned `Optionals` a handful of times throughout these lectures, but we haven't formally covered them just yet. Now is the time. `Optionals` let us have a spot for information that may or may not exist. We have used types such as `Int` to hold values since the first lesson. For example, what if I needed to store an age property for my uses, what am I supposed to do if I don't know someone's age?

You may decide to use a value like `0`, but then you run into the issue of getting confused between newborns and people whose age you don't know. Potentially, you may try to use a special number like `-1` or `1000`. Even though those aren't ever going to be someone's age, you run the risk of having oddities and inconsistencies in your code if you do that.

Swift has a solution to this dilemma called `Optionals`, you can make `Optionals` out of any type. An `Optional Int`, for example, might have a value of `20`, or `400`, but it might not have a value at all. When we have no value in an `Optional` typed place, we can say the value is `nil` in Swift.

To make a type `Optional`, all we have to do is add a question mark after its type declaration. For example, we can make an `Optional Int` like this:

```
var myOptionalNumber: Int? = nil
```

This `myOptionalNumber variable` is an `Int?` or an `Optional Int`, this means it may or may not represent a value. Specifically, in this case, we set it equal to `nil` or no value. But now, maybe it is later, and I know what the value is, nothing changes with how I can assign a value to this `variable`, I would just assign it an `integer` like any other `variable`.

```
myOptionalNumber = 60
```

As I previously said, we can have optionals of any type. For example, we might have an `Optional [String]` It might store multiple `Strings` or not exists at all.

```
var names: [String]? = nil
```

Now that we have this `Optional [String]` what would happen if I was to try to get the count on names? A non `Optional Array` has a property on it called `.count` that stores how many items are in the `Array`. But, our names `Array` is `nil`, it is nothing, not an `Array`, so it doesn't have a count.

Because of this situation, if we try to read `.count` it is unsafe, and could cause a program crash. We cannot read from something that doesn't exist right? So in order to do that, we first need to establish if the value exists, then if it does, we can read from it. Or in other words, we need to see if our `Optional` value is `nil`. This process of checking if our `Optional` contains a value is called unwrapping.

Optional Binding

There are several common ways to unwrap `Optionals`, the first of which being an `if let` or Optional Binding. `if let` attempts to unwrap our value, and if it is not `nil` we can do a block of code using the unwrapped value. If the value is nil, then the block of code simply wouldn't run.

```
if let safeNames = names {
     // if names is not nil, then we enter this if statement and can
reference
     // its value via the safeNames constant.
     print(safeNames.count)
} else {
     print("The Value Is Nil")
}
```

What the above code is really saying is: "If `names` is not `nil`, store the value inside `safeNames`. Then enter the scope of my if statement, so I have access to this `safeNames` constant."

Something you may have seen before is when you print out a value it prints something like this: `Optional(60)`. This is an optional, that has a value, but hasn't been unwrapped. Let's make an example to show this off, and to get more practice.

```
var optionalName: String? = "Mawile"
print(optionalName) // Expression implicitly coerced from `String?` to
`Any`
```

This printed out `Optional("Mawile")`. This is because we are trying to print out an optional that has a value, without unwrapping it. If it didn't have a value it would print out `nil`. So if we wanted to print of the name, we need to unwrap it first. Let's do that using our `if let` syntax.

```
if let optionalName {
     print(optionalName)
     // optionalName = "Vulpix" | optionalName is a let constant.
}
```

In certain situations, we can choose to not rename our optional value, and instead just put `if let <NameOfVariable>`. However, this is still doing the same thing as before, where it is making a new constant. This means we cannot access the original variable now with this situation, where if we decided to use the other method, we could still access the original variable.

```
if let name = optionalName {
    print(name)
    optionalName = "Vulpix"
}
```

We didn't put an else here, because we simply don't have to. In this case, if the value is `nil` I just don't care and don't want to run any code. If we run this code we will see that it prints out our name without any weird `Optional` stuff around it. But if `optionalName` was nil, nothing would be printed at all.

Previously in the error handling lesson, we talked about `guard` statements. `guard` statements let us take the sad route first, and then make sure that in the rest of our scope, that the happy route is the only option. In the case of `Optionals`, we can use `guard` statements to unwrap them, or take the sad route. When we use a guard like this, we call it a `guard let`. The guard let will check if our value is nil, and if it is, it will exit the current scope. This means that if it succeeds, then in the rest of the scope we can refer to the unwrapped value!

To show off the use of guard statements, we need to be typically in a function of some sort, so let's do our classic greet function to show this off. In this case, we are going to take an optional parameter that will be their name, and if we don't know their name, we can greet them by asking what their name is.

`Optionals` are just a type. When we make another type `Optional`, is actually saying that we are making an `Optional` that can hold a certain type. Kind of like a collection. Where we can create an `Array` that holds `Strings`, we can make an `Optional` that holds an `Int`. We can treat `Optionals` like all of our other types, We can have functions take them as parameters, and/or return them. We can have stored or computed properties that are `Optional`. We can make an `Array` of `Optionals` if we want. I cannot stress this enough, that we can make any Type `Optional`, whether that be one we create or an existing one, and we can use one of these `Optional` types anywhere we can use any of our other Types.

```
func greet(person name: String?) {
    guard let personsName = name else {
        print("Hi, what is your name?")
        return
    }
```

```
    print("Hello, \(personsName)")
}
```

When we call a function like this, that takes an Optional parameter, We can pass a non-optional value as long as it matches the base type. Example: `greet` takes an `Optional String`, but we can pass it a regular `String`. However, non-optional parameters, cannot take optional values.

```
let myOptionalName: String? = "Jose"
let myNonOptionalName = "Lukas"

greet(person: myOptionalName)
greet(person: myNonOptionalName)
```

Nil Coalescing

Nil Coalescing is another way we can unwrap `Optionals`, we can even do it in a single line! The nil coalescing operator, represented by the `??` unwraps an `Optional` and returns the value inside if there is one. If there is not a value, so if the `Optional` is `nil`, then a provided default value is used instead. This means that either way the result will have a value, so it isn't going to be optional.

To show this off, let's start by making a function that can return an `Optional` value.

```
func pokemon(selection: Int) -> String? {
    return switch selection {
        case 1: "Bulbasaur"
        case 2: "Charmander"
        case 3: "Squirtle"
        // If selection is not 1, 2, or 3 we will return nil.
        default: nil
    }
}
```

When we call this function, we may get back `nil` as it returns an `Optional`. We may not want to deal with the `Optional` and instead just use a default value if that happens, this is where nil coalescing comes in. When we use the `??` nil coalescing operator, we give a default value to use if our first option is `nil`.

```
let myStarter = pokemon(selection: 12) ?? "Pikachu"
// 12 is invalid so our function returns nil, using the nil coalescing
operator
```

```
// means that we will use the default value instead.
print(myStarter) // Pikachu
let mySecondStarter = pokemon(selection: 1) ?? "Pikachu"
print(mySecondStarter) // Bulbasaur
```

We have used the concept of providing a default value in place of a nil value before. Back when we first covered dictionaries, we discussed how if there is no a matching key value pair for the key we provide to a dictionary, it will return `nil`. There is a way to provide a default value to use in the situation so that you don't have to deal with an `Optional`. I'm sure you can draw the similarities. Let's do an example:

```
let raceAndMiles = [
    "5k": 3.1,
    "10k": 6.2,
    "marathon": 26.2
]

let halfMarathon = raceAndMiles["halfMarathon", default: -1] // -1
let ironMan = raceAndMiles["ironMan"] ?? -1 // -1
let marathon = raceAndMiles["marathon", default: -1] // 26.2
let tenK = raceAndMiles["10k"] ?? -1 // 6.2
```

Force Unwrapping

There is another way that we can unwrap `Optionals`, but it is dangerous. This method is called **Force Unwrapping**. You should only use this if you are positive there is a value; however, I typically avoid it all together. If you force unwrap an `Optional` value that is `nil` your program will crash, and no one wants that.

We can cause a force unwrap by using a `!` bang after the value we want to force unwrap.

```
let someOptionalNumber: Int? = 20
let forcedNumber = someOptionalNumber!
// Since we force unwrap forcedNumber is not an Optional.
```

Optional Chaining

Optional Chaining is a process for checking properties, and calling methods or subscripts, on an optional that might currently be `nil`. If the optional contains a value, the property, methods, or subscripts calls succeed. If the optional is `nil` the property, method, or subscript call will not execute resulting in a `nil` value if a value was expected. We can chain multiple of these queries together, this entire chain will fail gracefully (won't crash) if any link in the chain is `nil`.

```
struct Device {
    let type: String
    var price: Double
    let color: (red: Int, green: Int, blue: Int)
}
```

When we are creating a variable or constant that is `Optional` we can choose to omit assigning it a value completely, this would result in the variable or constant being `nil`.

```
var myPhone: Device? // nil

// To use optional chaining we put a `?` before the `.` when accessing
the
// property or method.
let devicePrice = myPhone?.price

if let devicePrice {
    print("My price is $\(devicePrice)")
}
```

Normally, it would be unsafe to access a property like count on a string that may not exist. So normally we would have to unwrap it first to see what is inside. By using optional chaining we can avoid this hassle, we can also combine optional chaining with one of our existing methods for unwrapping.

```
var name: String?

if let unwrappedName = name {
    print("name contains \(unwrappedName.count) letters")
} else {
    print("name contains 0 letters")
}
```

We could rewrite the above using nil coalescing and optional chaining like this:

```
print("name contains \(name?.count ?? 0) letters.")
```

Implicitly Unwrapped Optionals

Implicitly Unwrapped `Optionals` can be used when you know the optional will contain a value by the time it is being used. They behave as if they aren't `Optionals` at all; however, they can store a `nil` value. Using an Implicitly Unwrapped `Optional` saves you from

having to use any of the tricks we have covered previously; at the risk of crashing if when you try to use the value it is still `nil`. As a rule I would stick to using normal optionals, but these can be useful in the right situation.

To create an implicitly unwrapped optional, we simply replace where we would put the `?` normally to create an optional with a `!`.

```
// Stores a nil value
var howOld: Int! = nil

// If this line was missing, our app would crash as we would then
// be failing to honor our promise that the `howOld` constant would
// have a value before being used.
howOld = 5

// Able to use without any unwrapping.
if howOld > 18 {
    print("Adult")
} else {
    print("Child")
}
```

Unwrapping Multiple Values at Once

We can use `if let` and `guard let` to unwrap multiple `Optionals` at once. This just means that in order for them to succeed, all the `Optionals` have to have non `nil` values.

```
var x: Int? = 5
var y: Int? = 2

if let x = x, let y = y {
    print(x + y)
}

func addTwoOptionals(_ opOne: Int?, _ opTwo: Int?) -> Int? {
    guard let unwrapOne = opOne,
          let unwrapTwo = opTwo
    else { return nil }

    return unwrapOne + unwrapTwo
}

print(addTwoOptionals(x, y) ?? 0) // 7
```

Failable Inits

When we are making a struct or a class, and making `inits`, there may be times that we can't make an object for one reason or another. This could be due to bad information being given, or something that is out of our control. We can make what is called a fail-able initializer to handle cases like these. A fail-able `init` returns either the created instance or nil if the instance cannot be created. This means that it is creating an `Optional` version of our type when we call one.

```
struct Person {
    let name: String
    let age: Int

    // We make an optional init by adding a `?` after the `init`
keyword.
    init?(named personsOptionalName: String?, age personsOptionalAge:
Int?) {
        guard let unwrappedName = personsOptionalName,
              let unwrappedAge = personsOptionalAge
        else { return nil }

        self.name = unwrappedName
        self.age = unwrappedAge
    }
}

let teacher = Person(named: "Lukas", age: 22)
// Teacher is a Person? since our Person init is failable.
```

Typecasting

Typecasting is when we cast one type to another. Or basically we check if something is a certain type, and if it is, we can treat it like it's that type. Usually this will go along with stuff like protocols and class inheritance.

Let's say I have a protocol `Shape` and a few different structs that conform to that protocol.

```
protocol Shape {
    var numberOfSides: Int { get }
}

struct Triangle: Shape {
    let numberOfSides = 3
```

```
}

struct Hexagon: Shape {
    let numberOfSides: Int = 6
    var description = "Hexagons are the Bestagons"
}

struct Circle: Shape {
    let numberOfSides: Int = 0
    let diameter: Int
}
```

We could now take any instances of these structs that conform to this `Shape` protocol and store them together in an `Array` of `Shape`. Or really just store them anywhere that we can store a `Shape`. Some of these structs, however, contain some extra functionality that isn't provided by the `Shape` protocol, so we wouldn't have access to that information. This is where Typecasting comes in. We can use typecasting to treat one type like another, when it has been generalized, like we are doing here. Let's show this off.

We can type cast in a safe manor by using the `as?` keyword followed by the Type we want to cast to. So if I have a `Shape` I can attempt to cast it to a `Hexagon` for example by using its variable or constant name followed by `as? Hexagon`. This will result in an `Optional Hexagon`, but typically we immediately unwrap this optional, so we end up with something like the following:

```
let myShapes: [Shape] = [Triangle(), Hexagon(), Circle(diameter: 10)]

for shape in myShapes {
    if let hexagon = shape as? Hexagon {
        // Since we cast the shape, we get access to the unique
        // properties and methods of the casted to type.
        print(hexagon.description)
    } else if let circle = shape as? Circle {
        print(circle.diameter)
    } else {
        print(shape.numberOfSides)
    }
}
```

Optionals Review

1. Optionals let us represent the absence of a value in a clear and unambiguous way.
2. Swift won't let us use optionals without unwrapping them, safely or unsafely.

3. You can force unwrap optionals with an exclamation mark, but if you try to force unwrap a `nil` value, your program will crash.
4. Implicitly unwrapped optionals don't have the safety checks of regular optionals. `let x: Int! = nil` but crash if we access them when they don't have a value.
5. You can use nil coalescing to unwrap an optional or provide a default value if that value was `nil`: `??`
6. Optional chaining lets us write code to read or manipulate an optional value, but if the optional value turns out to be empty the code isn't ran, and it returns `nil`. `foo.func()?.otherFunc()`
7. You can use `try?` to convert a throwing function into an optional return value, resulting in `nil` if an error is thrown. You can also use `try!` to crash if an error is thrown, but avoid the optional.
8. If you need your initializer to be able to fail when given bad input, use `init?()` to make a fail-able initializer. This will result in a `nil` value if the object cannot be created.
9. You can use type casting to convert one type of object to another type.